

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	S. Muskan	Reg. No.:	192472182
Section / Batch:		Date:	

Code of Conduct

I, **S. Muskan, (192472182)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **S. Muskan**

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.
2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing. Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.
3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques. Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.
4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames. Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.
5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing. Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

1. CFG Trees vs Dependency Parsing

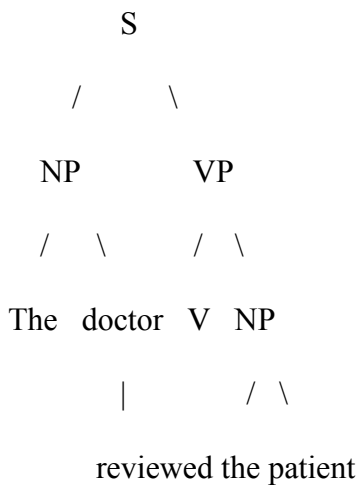
Both CFG trees and dependency parsing represent sentence structure, but they focus on different aspects.

CFG Trees

A **Context-Free Grammar (CFG)** represents a sentence using hierarchical constituents such as **NP (Noun Phrase)** and **VP (Verb Phrase)**.

Example:

“The doctor reviewed the patient.”

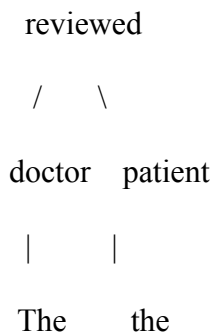


The main strength of CFG is that it clearly represents **phrase structure** and shows how words combine to form larger grammatical units.

However, CFG trees may require several levels of intermediate nodes before the actual relationship between two words becomes clear.

Dependency Parsing

Dependency parsing directly represents relationships between words using **head–dependent relations**.



Here:

reviewed → doctor (subject)

reviewed → patient (object)

The verb “**reviewed**” is the central head, while “doctor” and “patient” depend on it.

Comparison

Aspect	CFG Tree	Dependency Parsing
Main representation	Constituents/phrases	Word-to-word dependencies
Focus	Phrase structure	Relationships between words
Hierarchy	Multiple phrase levels	Direct head-dependent links
Subject/object relationships	Indirect through tree	Directly represented
Useful for	Grammar and syntactic structure	Semantic relations and dependency analysis
Long-distance relationships	Can become complex	Often easier to represent

Justification

For **grammar checking**, CFG is useful because it represents whether words form valid grammatical constituents.

However, if the requirement is specifically to **capture relationships between words**, dependency parsing is more effective.

For example:

Doctor → reviewed → patient

immediately identifies who performed the action and who received it.

Conclusion: CFG is better for constituent-based grammatical structure, while **dependency parsing is more effective for directly capturing relationships between words**. For modern NLP applications requiring grammatical relations, dependency parsing is generally the stronger choice.

2. Top-Down Parsing vs Earley Parsing

The scenario requires a parser that can handle **incomplete and ambiguous input efficiently in real time**.

Top-Down Parsing

Top-down parsing begins with the start symbol and attempts to generate the input.

S

↓

VP

↓

V + NP

↓

...

It predicts what the sentence should look like before matching the input.

Problem with Dynamic Input

Consider a user entering:

“Book a flight...”

The system does not yet know the complete structure.

If the user continues:

“...to Delhi with a window seat.”

the parser may need to explore alternatives and potentially backtrack.

This can increase latency.

Earley Parsing

Earley parsing maintains a **chart of parsing states** and uses three major operations:

Prediction → Scanning → Completion

It can maintain partial structures while processing the input.

For example:

Book

↓

Book a

↓

Book a flight

↓

Book a flight to Delhi

↓

Book a flight to Delhi with a window seat

The parser does not have to discard all previous work whenever new information arrives.

Comparison

Aspect	Top-Down	Earley
Starting approach	Start symbol → input	Maintains input parsing states
Backtracking	Can be high	Reduced through chart/state storage
Ambiguity	Can require repeated exploration	Can represent multiple possibilities

Incomplete input	Less suitable	Better suited
General CFG	Depends on grammar/parser	Handles general CFG
Dynamic input	Less efficient	More suitable
Real-time use	Can suffer from latency	Better for incremental processing

Technical Reasoning

The important difference is not simply that Earley is “faster.” Its advantage comes from **avoiding repeated work by storing intermediate states**.

For an ambiguous command such as:

“Book a flight to Delhi with a window seat”

the parser can preserve possible interpretations until sufficient information is available.

Conclusion

For a system receiving **incomplete, ambiguous, and continuously arriving input**, **Earley parsing is more suitable than conventional top-down parsing** because it supports partial parsing and manages ambiguity without repeatedly restarting the analysis.

3. CFG vs PCFG vs Neural Parsing

Sentence:

“She saw the man with a telescope.”

This sentence is ambiguous because “**with a telescope**” can modify either “**saw**” or “**the man.**”

Interpretation 1 – She used a telescope

She

\

saw

/ \

man with a telescope

Meaning:

She used a telescope to see the man.

Interpretation 2 – The man had a telescope

She

\

saw

\

man

\

with a telescope

Meaning:

She saw a man who had a telescope.

CFG

CFG can generate multiple valid parse trees.

For example:

$S \rightarrow NP VP$

$VP \rightarrow V NP$

$VP \rightarrow VP PP$

$NP \rightarrow NP PP$

Both interpretations can therefore be syntactically valid.

Limitation

CFG does not inherently tell us **which interpretation is more likely**.

So it handles structural ambiguity but does not effectively resolve it using probability or context.

PCFG

A **Probabilistic CFG** associates probabilities with grammar rules.

For example:

$VP \rightarrow VP PP$ [0.65]

$NP \rightarrow NP PP$ [0.35]

The parser can compare candidate trees and select the one with the higher probability.

Therefore:

Parse 1 $\rightarrow$ 0.70

Parse 2 $\rightarrow$ 0.30

The higher-probability interpretation can be selected.

Advantage

PCFG provides a principled way to **rank competing CFG parses**.

Limitation

Its decision depends heavily on the quality and representativeness of the training data used to estimate probabilities.

Neural Parsing

Neural parsers use learned representations from large datasets and can consider broader contextual information.

For example, the surrounding context could determine whether the telescope belongs to the woman or the man.

Consider:

“She saw the man with a telescope. The telescope was in her hand.”

A neural parser can use this contextual information to favor the interpretation that **she used the telescope**.

Comparison

Aspect	CFG	PCFG	Neural Parsing
Generates syntactic structures	Yes	Yes	Yes
Handles structural ambiguity	Yes	Yes	Yes
Ranks alternatives	No	Probability-based	Learned scoring
Uses context	Limited	Limited	Stronger
Data requirement	Low	Training corpus for probabilities	Large training data
Interpretability	High	High	Lower
Real-world adaptability	Limited	Moderate	High

Technical Reasoning

CFG answers:

“Which structures are grammatically possible?”

PCFG answers:

“Which grammatical structure is statistically more likely?”

Neural parsing can go further:

“Which interpretation is most appropriate considering learned linguistic patterns and contextual information?”

Conclusion

For a **real-world NLP system**, neural parsing is generally the most effective of the three for ambiguity resolution because it can exploit richer contextual information.

However, **PCFG remains useful when interpretability and probabilistic grammatical analysis are important**, while CFG provides the fundamental syntactic framework.

Therefore:

CFG → possible structures

PCFG → probable structure

Neural → context-aware structure

4. Feature Structures vs Subcategorization Frames

The system must handle two related but different requirements:

1. **Subject–verb agreement**
2. **Verb argument structures**

These are not exactly the same problem.

Feature Structures

Feature structures represent grammatical properties associated with linguistic elements.

For example:

Subject:

[CAT: NP

PERSON: 3

NUMBER: SINGULAR]

Verb:

Verb:

[CAT: V

PERSON: 3

NUMBER: SINGULAR]

A grammatical constraint can be expressed as:

Subject.NUMBER = Verb.NUMBER

Subject.PERSON = Verb.PERSON

Thus:

“The doctor recommends medication.”

is valid because both subject and verb are singular.

But:

“The doctor recommend medication.”

violates the agreement requirement.

Strength

Feature structures are therefore highly effective for **enforcing grammatical agreement constraints**.

Subcategorization Frames

Subcategorization describes the arguments that a particular verb requires or permits.

For example:

recommend

→ Subject + Object/Complement

In the healthcare scenario:

“The doctor recommends starting medication.”

The frame can be represented as:

RECOMMEND

└— Subject → Doctor

└— Complement → Starting medication

Similarly:

review

└— Subject → Doctor

└— Object → Patient

This tells the parser which entities should participate in the verb's action.

Comparison

Aspect	Feature Structures	Subcategorization Frames
Subject–verb agreement	Excellent	Not primary purpose
Number/person	Explicitly represented	Usually not the main focus
Grammatical constraints	Strong	Limited
Verb arguments	Can represent features	Excellent
Verb-specific requirements	Limited	Strong
Example	Doctor = singular	Recommend → subject + complement

Technical Reasoning

Suppose the sentence is:

“The doctor recommends medication.”

Feature structures verify:

doctor.NUMBER = singular

recommends.NUMBER = singular

Subcategorization verifies:

recommend

→ subject = doctor

→ complement = medication/action

Therefore, the two approaches **complement each other rather than being direct substitutes**.

Justification

If the question specifically asks which is better for **enforcing grammatical correctness**, **feature structures are the stronger choice**, because they explicitly encode constraints such as number and person agreement.

If the requirement is to determine **what arguments a verb requires**, subcategorization frames are better.

Conclusion

Feature Structures

↓

Agreement + grammatical constraints

Subcategorization Frames

↓

Verb + argument relationships

For the stated requirement of **subject–verb agreement and grammatical correctness**, feature structures provide better direct support, while subcategorization frames should be added for accurate verb argument analysis.

5. Transition-Based vs Graph-Based Dependency Parsing

The requirement is:

Fast and accurate dependency parsing on large datasets.

The two approaches make dependency decisions differently.

Transition-Based Parsing

Transition-based parsing constructs the dependency tree through a sequence of decisions.

A parser typically maintains structures such as:

Stack + Buffer

and performs transitions such as:

SHIFT

LEFT-ARC

RIGHT-ARC

REDUCE

For example:

Input:

The doctor reviewed the patient.

↓

SHIFT



SHIFT



RIGHT-ARC



...

Each decision determines the next transition.

Advantage

It is generally very fast because it processes the sentence through a sequence of local decisions.

Limitation

An early incorrect decision can affect later decisions.

Thus:

Early error



Wrong dependency



Later decisions affected



Potential error propagation

Graph-Based Parsing

Graph-based parsing considers possible dependency relationships and constructs a dependency graph.

Conceptually:

Words

↓

Possible dependency arcs

↓

Score candidate structures

↓

Select highest-scoring tree

For:

“The doctor reviewed the patient.”

the system evaluates relationships such as:

reviewed → doctor

reviewed → patient

doctor → The

patient → the

and selects the best-scoring dependency tree.

Advantage

Because it evaluates relationships more globally, graph-based parsing can achieve strong accuracy and is less dependent on a single sequence of local decisions.

Limitation

The larger search space can make graph-based parsing computationally more expensive.

Comparison

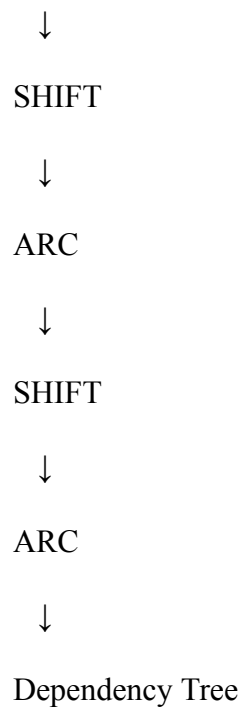
Aspect	Transition-Based	Graph-Based
Decision method	Sequential transitions	Global graph optimization
Processing	Local decisions	Global scoring
Speed	Usually very fast	Generally more computationally expensive
Accuracy	Can suffer from error propagation	Often strong global consistency
Memory/computation	Lower	Higher
Large-scale processing	Excellent for speed	More resource-intensive
Error propagation	Higher	Lower
Parallel/global analysis	Limited	Stronger

Technical Reasoning for Large-Scale Applications

Suppose a system needs to parse **millions of sentences**.

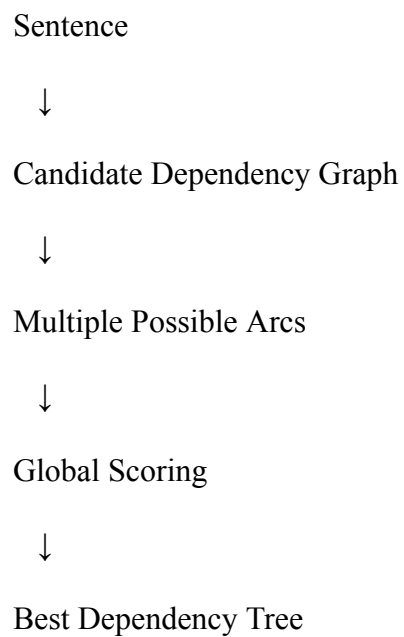
A transition-based parser can process sentences rapidly:

Sentence



Because the process consists of fast sequential decisions, it is highly suitable when **throughput and low latency** are priorities.

Graph-based parsing evaluates a larger set of possible structures:



This can provide better global decisions but requires more computation.

Justification

For the specific requirement:

“fast and accurate dependency parsing on big datasets”

the choice depends on which requirement dominates.

If **very high throughput and real-time performance** are the primary concerns, **transition-based parsing is more suitable**.

If **maximum structural accuracy and global consistency** are more important and computational resources are available, **graph-based parsing may be preferable**.

Final Recommendation

For a **large-scale production system where millions of sentences must be processed efficiently**, transition-based parsing is generally the more practical choice because of its speed and relatively low computational requirements.

Transition-Based



Fast decisions



High throughput



Large-scale NLP

Graph-based parsing can be selected when the additional computational cost is justified by the need for stronger global accuracy.

Overall Summary

Question	Alternatives	Best Choice for Given Scenario	Main Reason

1	CFG vs Dependency	Dependency Parsing	Directly captures word-to-word relationships
2	Top-Down vs Earley	Earley	Handles ambiguity and incomplete input efficiently
3	CFG vs PCFG vs Neural	Neural Parsing	Better contextual ambiguity resolution
4	Feature Structures vs Subcategorization	Feature Structures for agreement	Explicit grammatical constraints
5	Transition vs Graph	Transition-Based for large-scale speed	High throughput and low computational cost

Key idea: The correct technique depends on the **actual industry requirement**, not simply on which technique is more advanced. CFG provides structural foundations, probabilistic methods rank alternatives, feature structures enforce grammatical constraints, dependency methods expose word relationships, and neural methods provide richer contextual interpretation.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____